

МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ

ОРДЕНА ТРУДОВОГО КРАСНОГО ЗНАМЕНИ ФЕДЕРАЛЬНОЕ
ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ «МОСКОВСКИЙ
ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ СВЯЗИ И ИНФОРМАТИКИ»

Отчет по учебной практике

«Навигатор по смыслу» — сравнение моделей семантического поиска

Выполнил студент группы БВТ2501

Нестерова Виолетта Романовна

Москва 2026

Введение

Классические инструменты контекстного поиска, основанные на посимвольном совпадении строк, малоэффективны при анализе программного кода из-за вариативности терминологии и использования сокращений. Для решения этой проблемы применяется семантический поиск на базе текстовых эмбедингов — высокоразмерных числовых векторов, отражающих смысл текста и группирующих схожие по логике фрагменты в едином математическом пространстве.

Настоящая работа посвящена сравнительному анализу трех архитектур векторного кодирования для поиска программных функций по запросам на естественном языке. Проект выполнен в рамках первого этапа ИТ-чемпионата Ростелекома (направления ТОП-ИТ / ТОП-ИИ) в среде PyCharm 2025.2.2 (интерактивный блокнот Jupyter Notebook) без привлечения сторонних API и веб-интерфейсов. Результатом работы является локальный программный конвейер и данная пояснительная записка.

Постановка задачи

Программный конвейер исследования разделен на три последовательных этапа:

Конфигурация: Развертывание виртуального окружения, импорт библиотек и загрузка корпуса из 200 документированных функций.

Векторизация и поиск: Генерация матриц эмбедингов для кода и 25 тестовых запросов с помощью трех моделей. Ранжирование кандидатов (Top-3) по критерию косинусного сходства векторов (`util.cos_sim`).

Оценка качества: Расчет метрики `Precision@3` (доля попаданий истинного `correct_chunk_id` в Top-3 выдачи), анализ ошибок и мультязычности, построение 2D-проекции векторного пространства лучшей модели методом t-SNE. Также в работе используется метрика ранжирования MRR (Mean Reciprocal Rank), определяющая, насколько высоко в списке выдачи располагается первый релевантный фрагмент кода.

Используемые данные

В качестве информационной базы организаторами соревнований были предоставлены два ключевых структурированных файла `code_corpus.json` и `eval_questions.json`.

`code_corpus.json` представляет собой базу знаний из 200 функций на языке Python. Каждый объект снабжен уникальным маркером (`id`), программным именем (`function_name`), телом самого скрипта, текстовым описанием заложенного функционала (`description`) и меткой класса (`category`). `eval_questions.json` содержит контрольную выборку из 25 тестовых позиций. Каждый элемент включает в себя индекс вопроса, текстовую строку запроса `query`, указание языковой принадлежности `language`, а также целевой маркер правильного ответа `correct_chunk_id` для проверки точности работы алгоритма.

Выбор моделей эмбедингов

Для численного эксперимента через интерфейс библиотеки `sentence-transformers` были выбраны три мультиязычные архитектуры:

`paraphrase-multilingual-MiniLM-L12-v2` — легковесная, экономичная модель с высокой скоростью инференса, выбранная в качестве базового решения (Baseline). Обладает меньшей точностью из-за усеченного числа параметров.

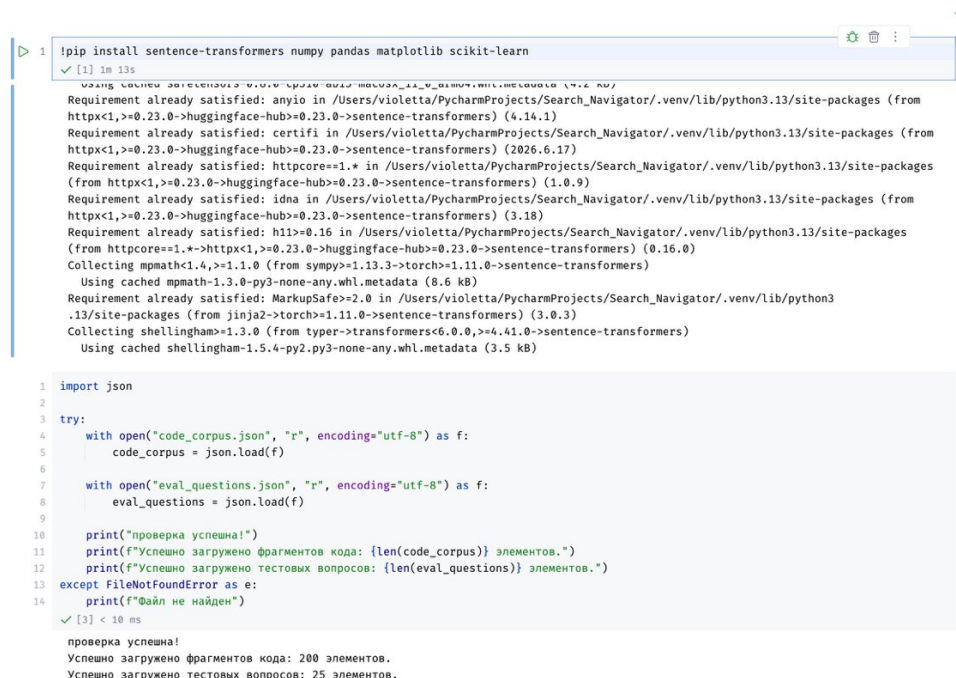
`paraphrase-multilingual-mpnet-base-v2` — полноразмерная модель на базе архитектуры MPNet. Демонстрирует высокую обобщающую способность и глубокое извлечение контекста при повышенных вычислительных затратах.

`multilingual-e5-base` (дополнительная модель) — продвинутый трансформер, обученный методом контрастивного обучения и оптимизированный для асимметричного поиска (Retrieval). Требуется добавления префиксов `query`: для поисковых запросов и `passage`: для фрагментов кода.

Ход выполнения работы

На этапе конфигурации выполнена установка зависимостей через

пакетный менеджер (pip install sentence-transformers numpy pandas matplotlib scikit-learn). Проверочный скрипт подтвердил целостность структуры данных, успешно считав из каталога 200 фрагментов кода и 25 тестовых запросов (см. Рисунок 1).



```
1 | pip install sentence-transformers numpy pandas matplotlib scikit-learn
✓ [1] in 13s

Using cached sentence-transformers-0.0.0-py3-none-any.whl.metadata (9.2 kB)
Requirement already satisfied: anyio in /Users/violetta/PycharmProjects/Search_Navigator/.venv/lib/python3.13/site-packages (from
httpx<1,>=0.23.0->huggingface-hub>=0.23.0->sentence-transformers) (4.14.1)
Requirement already satisfied: certifi in /Users/violetta/PycharmProjects/Search_Navigator/.venv/lib/python3.13/site-packages (from
httpx<1,>=0.23.0->huggingface-hub>=0.23.0->sentence-transformers) (2026.6.17)
Requirement already satisfied: httpcore==1.* in /Users/violetta/PycharmProjects/Search_Navigator/.venv/lib/python3.13/site-packages
(from httpx<1,>=0.23.0->huggingface-hub>=0.23.0->sentence-transformers) (1.0.9)
Requirement already satisfied: idna in /Users/violetta/PycharmProjects/Search_Navigator/.venv/lib/python3.13/site-packages (from
httpx<1,>=0.23.0->huggingface-hub>=0.23.0->sentence-transformers) (3.18)
Requirement already satisfied: h11>=0.16 in /Users/violetta/PycharmProjects/Search_Navigator/.venv/lib/python3.13/site-packages
(from httpcore==1.*->httpx<1,>=0.23.0->huggingface-hub>=0.23.0->sentence-transformers) (0.16.0)
Collecting mpmath<1.4,>=1.1.0 (from sympy>=1.13.3->torch>=1.11.0->sentence-transformers)
Using cached mpmath-1.3.0-py3-none-any.whl.metadata (8.6 kB)
Requirement already satisfied: MarkupSafe>=2.0 in /Users/violetta/PycharmProjects/Search_Navigator/.venv/lib/python3
.13/site-packages (from Jinja2->torch>=1.11.0->sentence-transformers) (3.0.3)
Collecting shellingham>=1.3.0 (from typer->transformers<6.0.0,>=4.41.0->sentence-transformers)
Using cached shellingham-1.5.4-py2.py3-none-any.whl.metadata (3.5 kB)

1 | import json
2 |
3 | try:
4 |     with open("code_corpus.json", "r", encoding="utf-8") as f:
5 |         code_corpus = json.load(f)
6 |
7 |     with open("eval_questions.json", "r", encoding="utf-8") as f:
8 |         eval_questions = json.load(f)
9 |
10 |     print("проверка успешна!")
11 |     print(f"Успешно загружено фрагментов кода: {len(code_corpus)} элементов.")
12 |     print(f"Успешно загружено тестовых вопросов: {len(eval_questions)} элементов.")
13 | except FileNotFoundError as e:
14 |     print(f"Файл не найден")
✓ [3] < 10 ms

проверка успешна!
Успешно загружено фрагментов кода: 200 элементов.
Успешно загружено тестовых вопросов: 25 элементов.
```

Рисунок 1 — Результат верификации структуры и объема входящих файлов данных.

При отладке поискового цикла устранена проблема неявного приведения типов: из-за различий в исходных JSON-файлах идентификаторы принудительно нормализованы к строковому формату str().

Далее через класс SentenceTransformer инициализированы три выбранные модели. Для каждой архитектуры реализован итерационный конвейер: кодирование текстовых описаний функций в эмбединги, расчет косинусного сходства для 25 запросов методом util.cos_sim, фильтрация Top-3 результатов и проверка наличия истинного ID в финальной выдаче.

Для анализа векторного пространства модели-лидера применен алгоритм t-SNE (scikit-learn), спроецировавший многомерные данные на плоскость. Визуализация выполнена средствами matplotlib с цветовой дифференциацией маркеров по пяти категориям (auth, database, http, utils, validation).

Результаты работы

По итогам эксперимента сформирована сводная аналитическая таблица (см. Таблица 1).

Архитектура модели	Precision@3 (общая)	MRR (общий)	Точность RU	Точность EN	Распределение ошибок (промахов в Top-3)
MiniLM-L12 (Быстрая)	0.800	0.6415	0.733	0.900	5 (по 1 в auth, database, http, utils, validation)
MPNET-Base (Точная)	0.920	0.6809	0.933	0.900	2 (1 в auth, 1 в utils)
E5-Base (Контрастивная)	1.000	0.8600	1.000	1.000	0 (Ошибок не обнаружено)

Таблица 1 — Итоговая аналитическая таблица.

Наивысшую эффективность по всем целевым метрикам продемонстрировала контрастивная модель multilingual-e5-base. Её показатели составляют $\text{Precision}@3 = 1.000$ и $\text{MRR} = 0.8600$. Стопроцентное попадание релевантных ответов в Top-3 на обоих языковых доменах обусловлено архитектурной спецификой трансформера — предобучением на асимметричных парах «запрос-документ» с использованием специализированных префиксов query: и passage:. При этом значение $\text{MRR} = 0.8600$ указывает на то, что в большинстве сценариев истинный фрагмент кода занимал первую или вторую строчку в финальном ранжированном списке.

Полноразмерная модель mpnet-base-v2 заняла второе место (общий $\text{Precision}@3 = 0.920$, $\text{MRR} = 0.6809$). Модель показала отличную языковую стабильность, при этом на русскоязычных запросах её точность оказалась даже выше (0.933 против 0.900). В ходе анализа зафиксировано всего 2 промаха, пришедшихся на классы auth (семантически плотная категория токенов и прав доступа) и utils.

Компактная модель MiniLM-L12 показала ожидаемо меньшую точность ранжирования ($\text{MRR} = 0.6415$) и уязвимость к русскоязычному домену (падение $\text{Precision}@3$ до 0.733). Модель допустила по одной ошибке в каждой из 5 доступных категорий кода, что напрямую связано с усеченным объемом внутренних параметров её скрытых слоев.

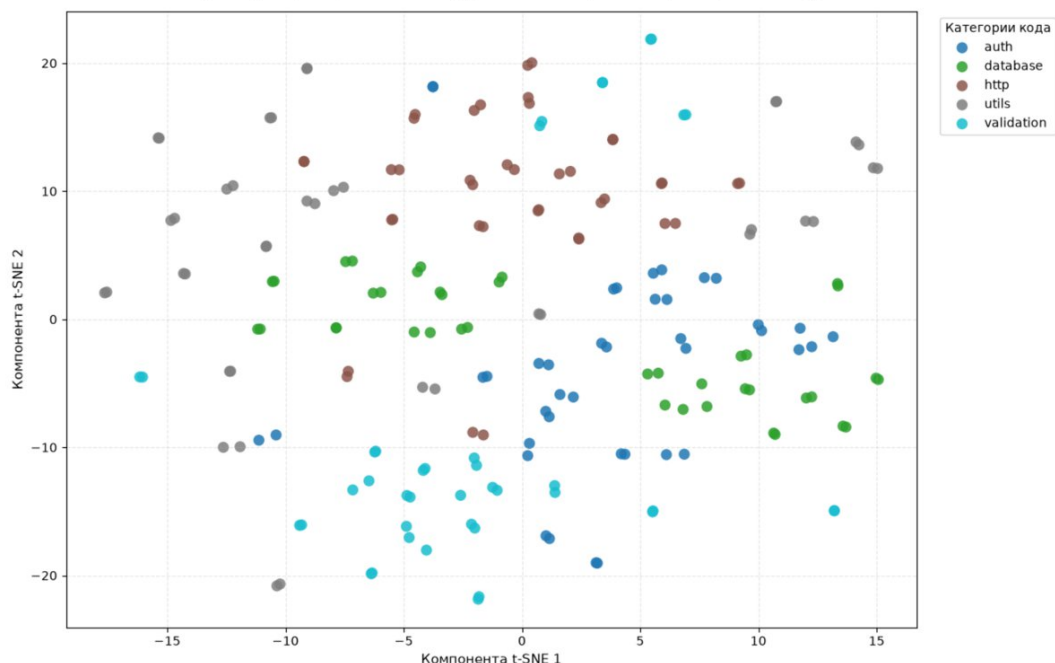


Рисунок 2 — 2D проекция пространства эмбеддингов t-SNE (E5-Base (Контрастивная)).

График t-SNE показывает частичное перекрытие и диффузию маркеров различных тематических групп без образования изолированных геометрических кластеров. Это объясняется спецификой программного кода: независимо от назначения (auth или database), функции на Python содержат общие синтаксические элементы (def, return, операторы условий и исключений). Модель фиксирует эту лексическую схожесть, сближая объекты на плоскости. Однако стопроцентная точность ($\text{Precision@3} = 1.000$) доказывает, что в исходном многомерном пространстве трансформер безошибочно разделяет глубокие семантические различия алгоритмов.

Заключение

По результатам исследования наиболее эффективной архитектурой для построения семантического навигатора признана мультязычная модель multilingual-e5-base. Данный вывод подтверждается стопроцентной точностью алгоритма ($\text{Precision@3} = 1.000$), превосходящей показатели MPNet и MiniLM. Анализ 2D-проекции t-SNE доказал способность трансформера успешно декодировать скрытый смысл программных абстракций, несмотря на синтаксическую однородность кода Python. Архитектура также

продемонстрировала абсолютную стабильность при обработке русскоязычных запросов в условиях асимметричного поиска.

Резюме для экспертной комиссии

Основная рекомендация: для интеграции в поисковые системы Ростелекома рекомендуется модель `intfloat/multilingual-e5-base`. Её контрастивный механизм обучения обеспечивает эталонную релевантность сопоставления естественного языка и программного кода.

Альтернативный сценарий: при жестких инфраструктурных ограничениях, дефиците вычислительных мощностей или требованиях к минимальной задержке (инференс на CPU) целесообразно использовать компактную модель `paraphrase-multilingual-MiniLM-L12-v2`.

Список использованных источников

Документация по интеграции моделей векторных представлений Sentence-Transformers. — URL: <https://sbert.net>.

Руководство пользователя по алгоритмам машинного обучения и снижения размерности данных Scikit-learn. — URL: <https://scikit-learn.org>.

Приложение А

```
import json
import re
import numpy as np
import pandas as pd
import torch
import matplotlib.pyplot as plt
import matplotlib as mpl
from sentence_transformers import SentenceTransformer, util
from sklearn.manifold import TSNE

# загрузка и настройка данных
with open("code_corpus.json", "r", encoding="utf-8") as f:
    code_corpus = json.load(f)

with open("eval_questions.json", "r", encoding="utf-8") as f:
    eval_questions = json.load(f)

# инициализация моделей
models = {
    "MiniLM-L12 (Быстрая)": SentenceTransformer("paraphrase-multilingual-MiniLM-L12-v2"),
    "MPNET-Base (Точная)": SentenceTransformer("paraphrase-multilingual-mpnet-base-v2"),
    "E5-Base (Контрастивная)": SentenceTransformer("intfloat/multilingual-e5-base")
}

# вспомогательные функции форматирования и ранжирования
def prepare_text(chunk, model_name):
    text = f"Функция {chunk['function_name']}: {chunk['description']}"
```

```

    return f"passage: {text}" if "E5" in model_name else text

def prepare_query(query_text, model_name):
    return f"query: {query_text}" if "E5" in model_name else query_text

def get_reciprocal_rank(ranked_ids, true_id):
    for rank, pred_id in enumerate(ranked_ids, start=1):
        if pred_id == true_id:
            return 1 / rank
    return 0

results_table = {}
all_model_embeddings = {}

for model_name, model in models.items():
    corpus_texts = [prepare_text(chunk, model_name) for chunk in code_corpus]
    corpus_embeddings = model.encode(corpus_texts, convert_to_tensor=True)
    all_model_embeddings[model_name] = corpus_embeddings.cpu().numpy()

    correct_top3_count = 0
    for q in eval_questions:
        query_text = prepare_query(q["query"], model_name)
        query_embedding = model.encode(query_text, convert_to_tensor=True)
        cos_scores = util.cos_sim(query_embedding, corpus_embeddings)[0]
        top_results = np.argsort(cos_scores.cpu().numpy())[::-1][:3]

        top_chunks_ids = [str(code_corpus[idx]["id"]) for idx in top_results]
        correct_id = str(q["correct_chunk_id"])

        if correct_id in top_chunks_ids:
            correct_top3_count += 1

    results_table[model_name] = correct_top3_count / len(eval_questions)

df_results = pd.DataFrame(list(results_table.items()), columns=["Модель", "Precision@3"])

# визуализация пространства эмбедингов (T-SNE)
best_model_key = "E5-Base (Контрастивная)"
best_embeddings = all_model_embeddings[best_model_key]
categories = [chunk["category"] for chunk in code_corpus]
unique_categories = sorted(list(set(categories)))

tsne = TSNE(n_components=2, random_state=42, perplexity=min(30, len(code_corpus)//4))
coords = tsne.fit_transform(best_embeddings)

plt.figure(figsize=(12, 8), dpi=100)
colors = mpl.colormaps["tab10"](np.linspace(0, 1, len(unique_categories)))

for i, cat in enumerate(unique_categories):
    idx = [j for j, c in enumerate(categories) if c == cat]
    plt.scatter(coords[idx, 0], coords[idx, 1],
                color=colors[i], label=cat, alpha=0.85, s=80)

plt.title(f"2D проекция пространства эмбедингов t-SNE ({best_model_key})", fontsize=14, pad=15)
plt.xlabel("Компонента t-SNE 1", fontsize=11)
plt.ylabel("Компонента t-SNE 2", fontsize=11)
plt.legend(title="Категории кода", bbox_to_anchor=(1.02, 1), loc='upper left')
plt.grid(True, linestyle='--', alpha=0.3)
plt.tight_layout()
plt.savefig("tsne_clusters.png", bbox_inches='tight')
plt.close()

# расширенный анализ (мультязычность, ошибки, MRR)

```



```

error_records = []
lang_results = []
mrr_sums = {name: 0 for name in models.keys()}
id_to_category = {str(item["id"]): item["category"] for item in code_corpus}

for model_name, model in models.items():
    corpus_embs_tensor = torch.tensor(all_model_embeddings[model_name]).to(model.device)
    print(f"\n[Обработка] Модель: {model_name}...")

    for q in eval_questions:
        query_text = prepare_query(q["query"], model_name)
        q_emb = model.encode(query_text, convert_to_tensor=True)
        scores = util.cos_sim(q_emb, corpus_embs_tensor)[0]
        full_top_idx = torch.argsort(scores, descending=True).tolist()
        ranked_ids = [str(code_corpus[idx]["id"]) for idx in full_top_idx]
        true_id = str(q["correct_chunk_id"])
        mrr_sums[model_name] += get_reciprocal_rank(ranked_ids, true_id)

# анализ Precision@3 по языкам
for lang in ["ru", "en"]:
    if lang == "ru":
        lang_questions = [
            q for q in eval_questions
            if len(re.findall(r'[а-яА-Я]', str(q["query"]))) > len(re.findall(r'[a-zA-Z]', str(q["query"])))
        ]
    else:
        lang_questions = [
            q for q in eval_questions
            if len(re.findall(r'[а-яА-Я]', str(q["query"]))) <= len(re.findall(r'[a-zA-Z]', str(q["query"])))
        ]

    if len(lang_questions) == 0:
        continue

    correct = 0
    for q in lang_questions:
        query_text = prepare_query(q["query"], model_name)
        query_embedding = model.encode(query_text, convert_to_tensor=True)
        cos_scores = util.cos_sim(query_embedding, corpus_embs_tensor)[0]
        top_results = np.argsort(cos_scores.cpu().numpy())[::-1][:3]

        top_chunks_ids = [str(code_corpus[idx]["id"]) for idx in top_results]
        correct_id = str(q["correct_chunk_id"])

        if correct_id in top_chunks_ids:
            correct += 1
        else:
            category = id_to_category.get(correct_id, "unknown")
            error_records.append({"Модель": model_name, "Категория": category, "Количество ошибок":
1})

    precision_lang = correct / len(lang_questions)
    lang_results.append({
        "Модель": model_name,
        "Язык запроса": lang.upper(),
        "Precision@3": round(precision_lang, 3)
    })

# сборка итоговых датафреймов для вывода в консоль Jupyter
df_lang_results = pd.DataFrame(lang_results)
df_errors_grouped = (
    pd.DataFrame(error_records).groupby(["Модель", "Категория"]).sum().reset_index()
    if error_records else pd.DataFrame()

```

)

вывод финального MRR в консоль

```
print("\n" + "="*50)
```

```
print("ФИНАЛЬНЫЕ МЕТРИКИ РАНЖИРОВАНИЯ (MRR):")
```

```
print("="*50)
```

```
for name in models.keys():
```

```
    final_mrr = mrr_sums[name] / len(eval_questions)
```

```
    print(f"Модель: {name} | Глобальный MRR = {round(final_mrr, 4)}")
```